

Fleet Management Backend

Peppermint Robotics — SDE-1 Hiring Challenge, **Assignment 2 (Backend)** — design walkthrough, written answers and system design. Stack: Node.js 20 · Express · ws · MQTT (Mosquitto) · MongoDB · Docker Compose.

What was built

A complete backend for a robot-fleet dashboard. Eight mocked robots replay their recorded telemetry from `events.jsonl` over MQTT — one dedicated OS process per robot, forked by a simulator supervisor. A Node.js backend consumes the stream, maintains the fleet's current state in a single in-memory store, and exposes that state two ways: a WebSocket push stream and a REST polling API, guaranteed consistent because both read the same object. Fleet history is persisted to MongoDB (the optional stretch goal). One command — `docker compose up --build` — starts the broker, database, backend and the robot fleet with no manual steps.

Fleet Management Backend — Assignment 2 (docker compose up)

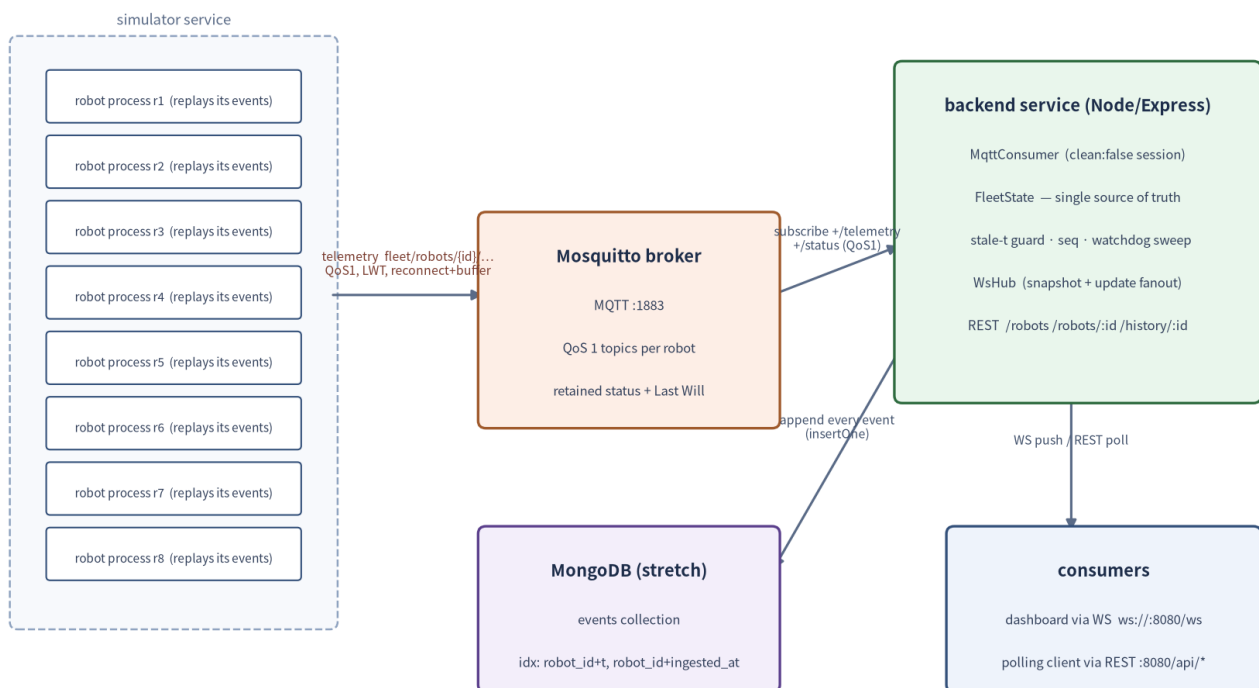


Figure 1 — services as wired by `docker-compose.yml`. The simulator forks 8 robot processes; each owns its own MQTT connection, Last Will and reconnect/buffer behaviour. The backend keeps one `FleetState` map that both the WebSocket hub and the REST routes read.

1. Running the system

```
docker compose up --build
# then, in another terminal:
curl http://localhost:8080/api/robots          # fleet snapshot (polling)
curl http://localhost:8080/api/robots/r1       # one robot + last-20 event tail
curl "http://localhost:8080/api/robots/history/r1?from=300&to=500" # Mongo history (stretch)
curl http://localhost:8080/api/health          # liveness + ingestion counters
# live stream:
node -e "new (require('ws'))('ws://localhost:8080/ws').on('message', d => console.log(d.toString()))"
```

Services and configuration

Service	Role	Port
broker	Eclipse Mosquitto 2 MQTT broker (anonymous, local demo only)	1883
mongo	MongoDB 7 — persists every ingested event (stretch goal)	27017
backend	Express REST + ws WebSocket + MQTT consumer + watchdog	8080
simulator	Supervisor; forks one robot process per entry in robots.json	—

Env var	Default	Meaning
REPLAY_SPEED	2	Playback multiplier; the 15-minute log replays in ~7.5 min, then loops
CHAOS	0	1 = robots randomly sever and restore their MQTT link (demos LWT offline detection)
STALE_MS	15000	Silence beyond this ⇒ watchdog flags the robot offline
MQTT_URL / MONGO_URI	compose defaults	Broker / database addresses

All images are pinned `linux/amd64` so the stack boots identically on the `x86_64` evaluation machine even when built on Apple Silicon.

2. Design walkthrough (where each decision lives)

Decision	File / function
One publisher process per robot (not 8 coroutines)	robot-simulator/src/index.js — <code>spawnRobot()</code> forks <code>robot-process.js</code>
MQTT topics, QoS 1, Last Will, reconnect+buffer	robot-simulator/src/robot-process.js — <code>mqtt.connect(will=...)</code>
Single source of truth for fleet state	backend/src/fleetState.js — class <code>FleetState</code> (Map keyed by <code>robot_id</code>)
Stale / out-of-order / duplicate protection	fleetState.js — <code>applyTelemetry()</code> drops events with <code>t</code> older than held
Offline detection	LWT via broker + <code>sweepStale()</code> watchdog (<code>STALE_MS</code>) in <code>index.js</code>
WS fanout with snapshot-on-connect	backend/src/wsHub.js — <code>broadcast()</code> + snapshot + ping/pong keepalive
REST polling API from the same store	backend/src/routes.js — <code>/robots</code> , <code>/robots/:id</code> , <code>/robots/history/:id</code>
History persistence (stretch goal)	backend/src/history.js — MongoDB, indexes on (<code>robot_id</code> , <code>t</code>)
Graceful degradation without Mongo	backend/src/index.js — history connects in background, retries forever

3. Written answers (ANSWERS.md, condensed)

Q1 — What holds the fleet's current state, and why that shape?

A single in-memory `Map<robot_id, record>` inside `FleetState` (`backend/src/fleetState.js`). Every consumer reads that one object: the MQTT consumer writes via `applyTelemetry()`, the WebSocket hub broadcasts the events it returns, and the REST endpoints serialize the same records via `snapshot()/getRobot()`. With one store there is no replication lag or second code path to drift — the assignment's consistency requirement ("a client using one should not see something inconsistent with the other") is guaranteed by construction. Updates are $O(1)$, snapshots $O(n)$ over a tiny n , and the flat record serializes straight to JSON for either transport. A per-robot monotonic seq plus a stale-t guard make late QoS-1 redeliveries harmless; each robot also keeps a 20-event tail ring for reconnecting clients.

Q2 — Tradeoff: MQTT with QoS 1, and reconciling it with the WebSocket fanout

I chose MQTT (Mosquitto) over HTTP callbacks, raw sockets, or Kafka/RabbitMQ. The problem is a textbook producer/consumer split over flaky links, and MQTT was designed for exactly that: it bought me (a) the **Last Will** — each robot registers a retained "offline" message at connect time, so the broker itself reports a dead robot within seconds; (b) **retained status topics** so a restarting backend immediately sees who is online; (c) **QoS 1 + a persistent consumer session** (`clean: false`) so messages published while the backend is down are redelivered. HTTP webhooks give none of this — every robot would need retry queues, and "dead" would be indistinguishable from "quiet". Kafka is absurdly heavy for ~2 msg/s and has no presence concept.

The cost: QoS 1 is at-least-once, so duplicates and reordering are normal — paid for by the stale-t guard and idempotent re-application in `applyTelemetry()`. And the WS fanout is fire-and-forget: an event can be applied to state while a dashboard client is disconnected, so that client misses the delta. I reconcile the two semantics by making the WS layer catch-up-based, not delivery-based: every client receives a full snapshot on connect (and on demand), and every record carries seq so gaps are detectable. Deliberately weaker than end-to-end exactly-once — the state store is the authority, the stream is a notification mechanism. Third cost: one more container to operate, though both sides reconnect automatically so a broker restart self-heals.

Q3 — What was left out, and what comes next

- **Auth/TLS** — anonymous Mosquitto listener and open REST/WS with permissive CORS; fine for local evaluation, unacceptable in production. First next step.
- **Richer history** — the stretch endpoint exists (Mongo, time-range query with limit cap) but has no aggregation, cursor pagination, or fleet-wide history.
- **Command path** — the system is telemetry-in only; a real operator eventually needs backend→robot commands (a `fleet/robots/{id}/cmd` topic).
- **Alerting** — error/blocked statuses are ingested and fanned out, but nothing pages anyone; a rules engine would subscribe to the same `onEvent` hook the WS hub uses.
- **Horizontal scaling** — one backend process holds all state; the sharding story is written up in §4 rather than built (ceremony at 8 robots).

4. System design questions (SYSTEM_DESIGN.md, condensed)

SD1 — Adding a feature later: does the design accommodate it?

Walkthrough: "alert the operator when any robot's battery drops below 20% while not charging." It plugs in without touching any existing component: the seam is the `onEvent` callback that `index.js` passes to `MqttConsumer` — today exactly one subscriber (`wsHub.broadcast`). An `AlertRules` module subscribes to the same hook, sees every validated/ordered event, and raises alerts as just another WS message plus a Mongo row via the existing `HistoryStore` connection. The general pattern: the pipeline is `producer → broker → consumer → state → fanout`, and each stage has one extension point. The one thing needing real rework is the reverse direction (robot commands), since the simulator currently only publishes.

SD2 — From 8 robots to 500: what breaks first?

Not the state store: a Map with $O(1)$ updates handles ~100 msg/s easily, and Mosquitto handles thousands of connections. First to break is the **full-snapshot payload + per-client WS fanout**: every event is written to every socket, and every connect/reconnect triggers a fresh ~150 KB snapshot — a reconnect storm stampede. Close behind: unbatched Mongo `insertOne` per event, and the simulator host (500 forked Node processes ≈ 15 GB RAM; demo-only). Fix order: subscription-aware/throttled broadcasts, cached-and-invalidated snapshot JSON, `insertMany` batching, then — only when a core saturates — MQTT shared subscriptions across N backend replicas with Redis pub/sub for cross-instance fanout, sharding `FleetState` by `robot_id` so each robot's state still lives in exactly one place.

SD3 — Bandwidth-limited robot↔backend links

- **Send deltas, not state** — heartbeats carry only `{robot_id, t, battery}`; `x/y/status` only beyond a deadband. `applyTelemetry` already tolerates partial updates structurally. ~60–80% traffic reduction.
- **Adaptive cadence** — idle/charging robots report every 30–60 s, moving/low-battery every 5 s; the watchdog then needs a per-robot expected interval (advertised in the retained status message) instead of one global `STALE_MS`.
- **Binary encoding** — CBOR/Protobuf over the same topics halves bytes again; resisted until deltas+cadence prove insufficient, because JSON debuggability is worth a lot at this scale.
- **Batch back-fill** — a robot reconnecting with a buffer sends one backlog message (array) instead of N publishes.
- **Never sacrificed**: the `t` timestamp and ordering guarantee — under constrained bandwidth late/out-of-order delivery gets worse, and the stale guard is the only thing between the dashboard and nonsense.

SD4 — A robot goes down mid-task

Detection is layered and all three layers exist in the code today. **(1) Broker Last Will**: if the TCP link dies ungracefully, the broker publishes the retained offline message — verified end-to-end by SIGKILLing a robot process during integration testing. **(2) Watchdog**: `sweepStale` flags any robot silent past `STALE_MS` (catches robots that never connected, or a dead broker). **(3) Semantic**: the log's own error/blocked/offline statuses surface a stuck-but-alive robot. The system then marks it offline in `FleetState` (both APIs reflect it immediately and WS clients get the transition pushed), and an orphaned mid-task robot — detectable via `status on_mission` plus an unpaired `task_started` — should have its task flagged for re-dispatch and its last position flagged for recovery. On recovery the robot reconnects, back-fills buffered reports, and the `t`-ordering guard files the backlog into history without ever moving the live position backwards.

SD5 — Slow/unreliable connection: late, out-of-order, missing updates

During the outage the robot's record simply stops advancing: position freezes at the last good value, `updated_at/last_seen` age visibly, and after `STALE_MS` the watchdog flips `online→false` with the last known state intact — a frozen robot never masquerades as a working one. **Out-of-order and duplicates** (normal under QoS 1 redelivery and buffered back-fill) are neutralized by the stale-`t` guard plus idempotent equal-`t` handling: the live

position is always the newest known, never rewind. The accepted cost: a slightly-out-of-order stream can drop a legitimate event that arrived after an even newer one — it self-corrects next tick, and the alternative (robots teleporting backwards on the map) is worse. **Recovery:** the robot's mqtt.js client reconnects with backoff and drains its offline buffer; the backend's persistent session receives anything the broker queued; the retained status re-announces `online:true`; the first fresh message flips online back. Known gap, stated plainly: events published while the broker and the backend's session were both down can be lost — closing it needs durable device-side storage with explicit replay ranges, for which the history endpoint already has the query shape.

5. Testing & verification evidence

Unit tests (18 passing) concentrate on the trickiest part — FleetState's ordering, dedup and watchdog semantics — plus the REST contract and the simulator's log loader:

```
backend:          15/15 passing (fleetState.test.js 11, routes.test.js 4)
robot-simulator: 3/3 passing (logLoader.test.js)
```

Beyond unit tests, the full stack was verified end-to-end in a live harness (real Mosquitto broker, backend, simulator, WebSocket client):

```
===== RESULTS =====
WS messages received: 194 | snapshot? true | updates: 193
REST robots online: 8 / 8
REST counters: {"ingested":282,"dropped_stale":0,"dropped_invalid":0}
r1 REST : {"x":579,"y":109.4,"battery":73.5,"status":"on_mission","seq":286}
r1 WS   : {"x":579,"y":109.4,"battery":73.5,"status":"on_mission","seq":286}
history (mongo down) status: 503 | ghost robot status: 404
REST/WS consistency (same store, seq monotonic): PASS

--- killing robot r5 process, expecting offline via LWT ---
r5 offline notification seen on WS: true (broker LWT)           offline detection: PASS
[supervisor] r5 exited (signal=SIGKILL); restart #1 in 2000ms
r5 recovered after supervisor restart: PASS (auto-recovered, fresh telemetry)

--- history store against a real (in-memory) MongoDB ---
all r1: 181 | range 300..400: 21 newest-first | limit cap ok | HISTORY STORE: PASS
```

6. Data provenance (honesty note)

The supplied events.jsonl reached me as a partial screenshot: t=0 fully legible, t=5 mostly legible (r7/r8 cut off), the remaining ~15 minutes not visible. data/events.jsonl therefore (1) reproduces the 14 legible screenshot lines verbatim, (2) infers r7/r8 @ t=5 using the log's own convention (idle: position unchanged, battery -0.1), and (3) extends the window to t=900 with a small seeded simulation (tools/gen-events.js, included and reproducible) that respects the obstacle rectangles measured from layout.png, drains battery while moving, recharges at pads, and emits a few rare task_started/task_completed extras exactly as the brief describes. This matches the brief's framing ("this log comes from a small scripted simulation... nothing hidden in it"). If the original file is available, drop it into data/ — nothing else changes. robots.json was likewise reconstructed from a screenshot; the picker/hauler types were illegible and are a reasonable guess.

7. Repository layout

```
fleet-backend/
■■■■ docker-compose.yml          # broker + mongo + backend + simulator, one command
■■■■ broker/mosquitto.conf       # anonymous local listener (documented tradeoff)
■■■■ data/
■   ■■■■ robots.json            # fleet roster (8 robots, reconstructed from screenshot)
■   ■■■■ events.jsonl           # 1,448 events, t=0..900, screenshot lines verbatim
■■■■ backend/
■   ■■■■ Dockerfile             # node:20-alpine, linux/amd64
■   ■■■■ src/
■   ■   ■■■■ index.js           # wiring: consumer → state → WS hub + REST + watchdog
■   ■   ■■■■ fleetState.js      # single source of truth (the trickiest part – tested)
■   ■   ■■■■ mqttConsumer.js    # QoS1 subscribe, persistent session, validation
■   ■   ■■■■ wsHub.js           # snapshot-on-connect, fanout, ping/pong, slow-client guard
■   ■   ■■■■ routes.js          # /health /robots /robots/:id /robots/history/:id
■   ■   ■■■■ history.js         # MongoDB persistence + time-range query (stretch goal)
■   ■■■■ tests/                 # fleetState.test.js, routes.test.js (15 tests)
■■■■ robot-simulator/
■   ■■■■ Dockerfile
■   ■■■■ src/
■   ■   ■■■■ index.js           # supervisor: forks one process per robot, restarts w/ backoff
■   ■   ■■■■ robot-process.js   # one robot: MQTT conn, LWT, replay loop, chaos mode
■   ■   ■■■■ logLoader.js       # filters events.jsonl per robot (tested)
```

```
■ ■■■ tests/                # logLoader.test.js (3 tests)
■■■ tools/gen-events.js      # seeded data reconstruction (provenance $6)
■■■ assets/layout.png        # site image from the assignment package
■■■ README.md                # run steps, decisions, AI delegation notes
■■■ ANSWERS.md               # the three backend written answers, unabridged
■■■ SYSTEM_DESIGN.md         # the five system design answers, unabridged
```

AI delegation: architecture (MQTT, process-per-robot, single source of truth, LWT+watchdog) was chosen by me; AI drafted boilerplate (Dockerfiles, compose, Express plumbing, test scaffolding) and the data-reconstruction generator under my direction. All design decisions, debugging (blocking Mongo connect, data-dir resolution, LWT test race) and written answers are reviewed and owned by me.